IoT Based Health Monitoring System (IHMS)

Final Capstone Project – Part I: Proposal and Prototyping

Student Name: MD. Kobir Hosen
Student ID: IT24030

Supervised by: Dr. Ziaur Rahman,
Professor, Dept. of ICT, MBSTU

Contents

1 Introduction **2**

2 Problem Statement **2**

3 Objectives **2**

4 Scope **2**

5 Stakeholders and Users **2**

6 Requirements **3**
6.1 Functional Requirements 3
6.2 Non-Functional Requirements 3

7 Proposed Hardware and Technology Stack **3**

8 Software Process Model: Agile (Scrum) **4**

9 Project Schedule **4**
9.1 Gantt Chart 4

10 Milestones **4**

11 Deliverables **5**

12 Risk Analysis **5**

13 UML Diagrams **5**
13.1 Data Flow Diagram (Level 0) 5
13.2 Data Flow Diagram (Level 1) 6
13.3 Use Case Diagram 6
13.4 Activity Diagram 7

13.5 Entity Relationship Diagram 8
13.6 Class Diagram 9
13.7 Object Diagram 9

14 Prototype **10**

15 Conclusion **10**

1 Introduction

Regular monitoring of vital signs is important for elderly people, patients with chronic diseases and people living far from hospitals. In most cases the measurements are taken manually and only during a hospital visit, so dangerous changes can go unnoticed. The **IoT Based Health Monitoring System (IHMS)** uses a wearable sensor node built with an ESP32 microcontroller, a heart rate and SpO_2 sensor (MAX30102) and a temperature sensor (DS18B20) to measure vital signs continuously and send them to a server. The server checks each reading, raises an alert when a value is abnormal and informs the patient and the assigned doctor immediately.

2 Problem Statement

- Vital signs are measured only occasionally, so sudden changes are missed.
- Doctors cannot see a patient's recent readings without a visit.
- Patients and families are not warned quickly when readings become dangerous.
- There is no central history of readings, alerts and prescriptions.

3 Objectives

1. Measure heart rate, body temperature and SpO_2 with an IoT sensor node and send them to the server.
2. Store every reading and show the latest values and history to the patient and doctor.
3. Compare readings with personal health limits and generate Medium, High or Critical alerts automatically.
4. Notify the patient and the doctor on every alert, acknowledgement and prescription.
5. Let the doctor review alerts and write prescriptions, and let the admin manage devices, limits and reports.

4 Scope

In scope: user login, device registration, sensor data collection, health limits, automatic alerts, notifications, doctor review and prescription, reports.
Out of scope (first release): ECG measurement, video consultation, SMS gateway, machine-learning based disease prediction.

5 Stakeholders and Users

Role	Description
Patient	Wears the device, views vitals, alerts and prescriptions
Doctor	Reviews alerts of assigned patients, writes prescriptions
Admin	Registers devices, assigns doctors, sets health limits, views reports
IoT Device	Sensor node that measures and sends the readings

6 Requirements

6.1 Functional Requirements

FR1 Users can log in and log out with role-based access.
FR2 The admin can register a device to a patient and assign a doctor.
FR3 The device sends heart rate, temperature and SpO_2 readings to the server.
FR4 The system validates and stores every reading with its time.
FR5 The patient can view the latest reading and the history of readings.
FR6 The system compares each reading with the patient's health limits and creates an alert (Medium, High or Critical) when needed.
FR7 The system notifies the patient and the doctor when an alert is created.
FR8 The doctor can view open alerts, acknowledge and resolve them.
FR9 The doctor can write a prescription (medicine and advice) for an assigned patient.
FR10 The admin can change the health limits and generate reports.

6.2 Non-Functional Requirements

Attribute	**Requirement**
Reliability	No more than 5% of readings lost; device retries if the network is down
Performance	An alert reaches the doctor within 10 seconds of an abnormal reading
Security	Passwords hashed; encrypted communication (HTTPS / MQTT over TLS); role-based access
Privacy	Medical data is visible only to the patient, the assigned doctor and the admin
Usability	The dashboard is simple enough for elderly users
Power	The wearable runs at least 24 hours on one battery charge
Maintainability	Modular, object-oriented code with documentation

7 Proposed Hardware and Technology Stack

Layer	**Technology**
Microcontroller	ESP32 (built-in Wi-Fi)
Sensors	MAX30102 (heart rate and SpO_2), DS18B20 (temperature)
Communication	MQTT or HTTP REST over Wi-Fi
Backend	Java (Spring Boot) or Python (Flask)
Database	MySQL
Frontend	HTML, CSS, JavaScript (Bootstrap) dashboard
Simulation	Wokwi / Tinkercad
Version control	Git and GitHub
Design tools	PlantUML, Overleaf

8 Software Process Model: Agile (Scrum)

Agile Scrum was chosen because hardware and software can be built and tested in small steps, requirements may change after sensor testing, and working increments (first the sensor node, then the server, then the dashboard) can be shown at the end of every sprint.

- Sprint length: 2 weeks; total duration: 12 weeks.
- Product backlog is built from the functional requirements.
- Each sprint ends with a demo, a review and a retrospective.

9 Project Schedule

Sprint	Weeks	Activities	Output
0	1–2	Proposal, planning, requirement analysis	Proposal, SRS
1	3–4	Circuit, system and database design; component purchase	UML diagram
2	5–6	Sensor node and ESP32 firmware	Device sends r
3	7–8	Server, database and MQTT/REST API	Readings store
4	9–10	Dashboard, alert module, doctor review and prescription	Full workflow
5	11–12	Integration testing, sensor calibration, deployment, documentation	Final system,

9.1 Gantt Chart

The Gantt chart is in `diagrams/gantt.puml` (rendered image `gantt.png`).

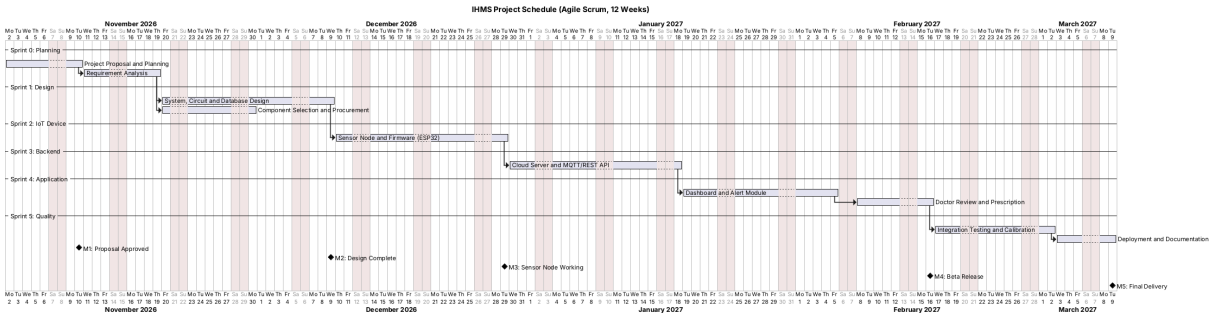


Figure 1: Project Gantt chart

10 Milestones

ID	Milestone	Week
M1	Proposal approved	1
M2	Design complete (diagrams and circuit)	4
M3	Sensor node working and sending data	6
M4	Beta release (alerts, doctor review, prescriptions)	10
M5	Final delivery	12

11 Deliverables

1. Project proposal (this document)
2. Software Requirement Specification (SRS)
3. UML diagrams (DFD, use case, activity, ERD, class, object)
4. Circuit diagram and bill of materials
5. ESP32 firmware source code
6. Server, database scripts and web dashboard source code
7. Working prototype
8. Test plan and test report
9. User manual and final project report

12 Risk Analysis

Risk	Level	Mitigation
Inaccurate sensor readings	High	Calibrate against a medical device; filter noisy values
Wi-Fi or server failure	Medium	Device buffers readings and resends later
Hardware delay or damage	Medium	Order spare parts early; test first in a simulator
Privacy of medical data	High	Encryption, access control, minimal data stored
Time shortage	High	Build core features first (reading, alert, notification)

13 UML Diagrams

All diagrams are PlantUML source files in the `diagrams/` folder. Their rendered images are shown below.

13.1 Data Flow Diagram (Level 0)

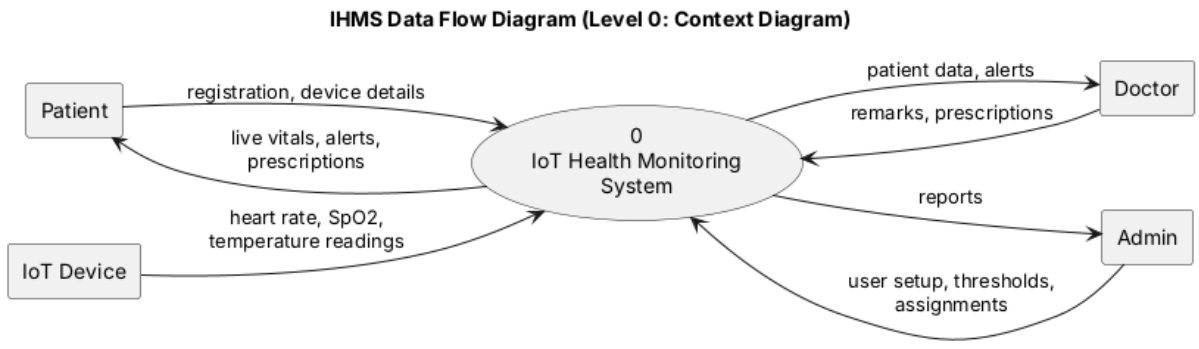


Figure 2: DFD Level 0 (context diagram)

13.2 Data Flow Diagram (Level 1)

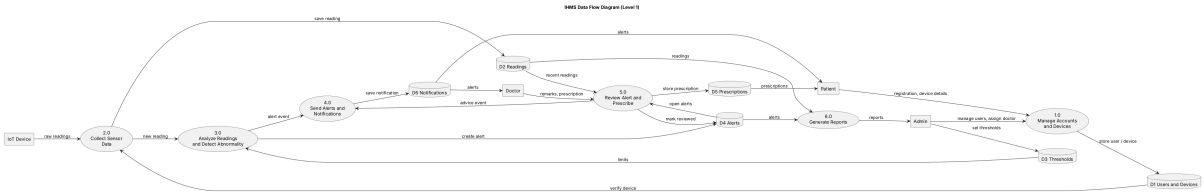


Figure 3: DFD Level 1

13.3 Use Case Diagram

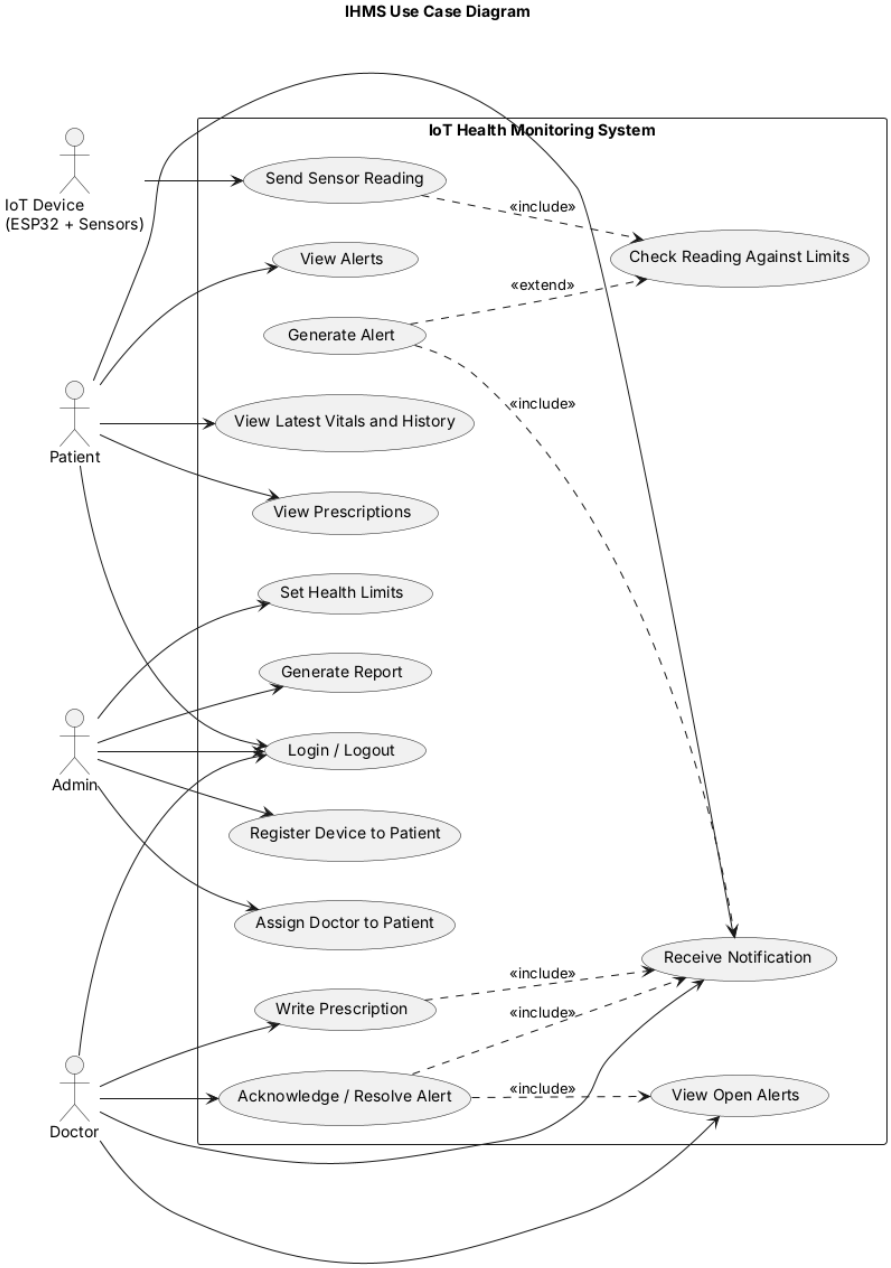


Figure 4: Use case diagram

13.4 Activity Diagram

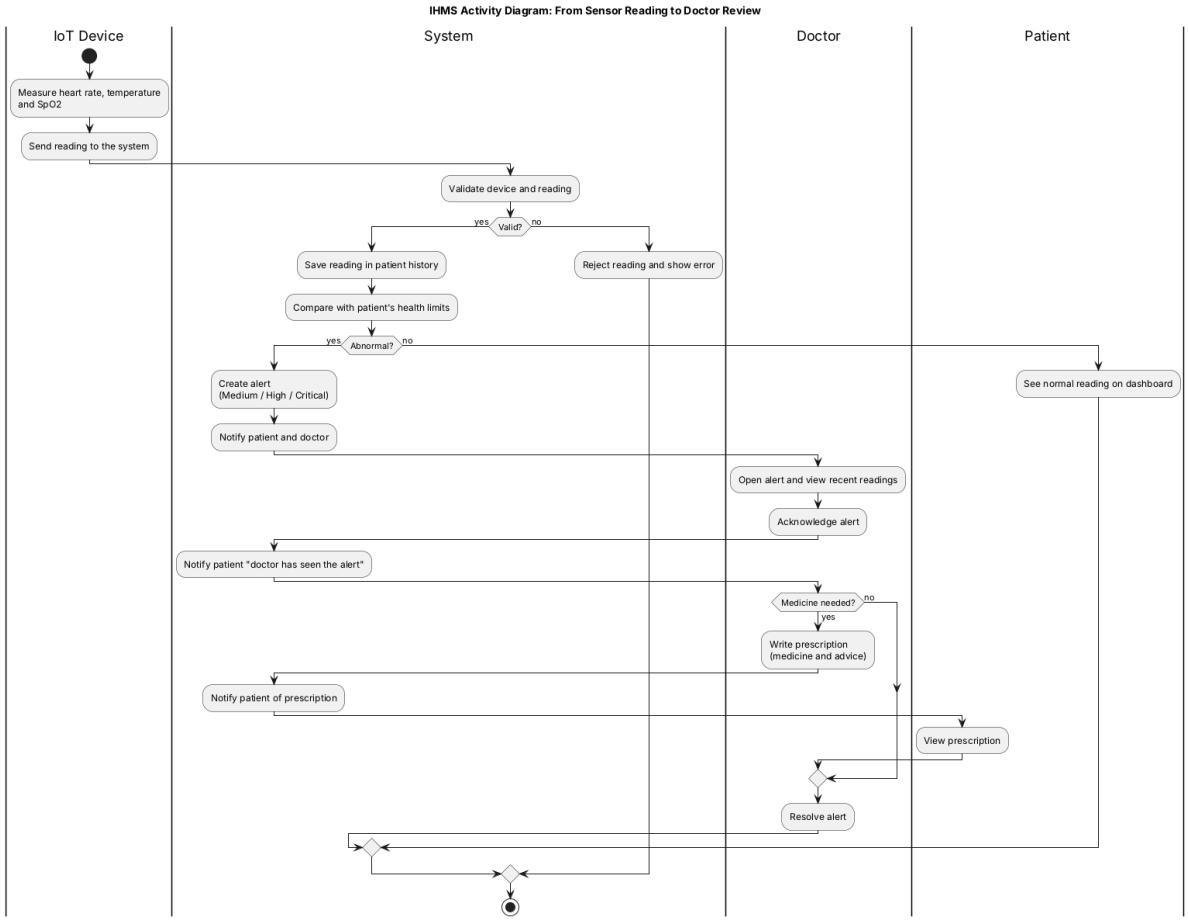


Figure 5: Activity diagram: from sensor reading to doctor review

13.5 Entity Relationship Diagram

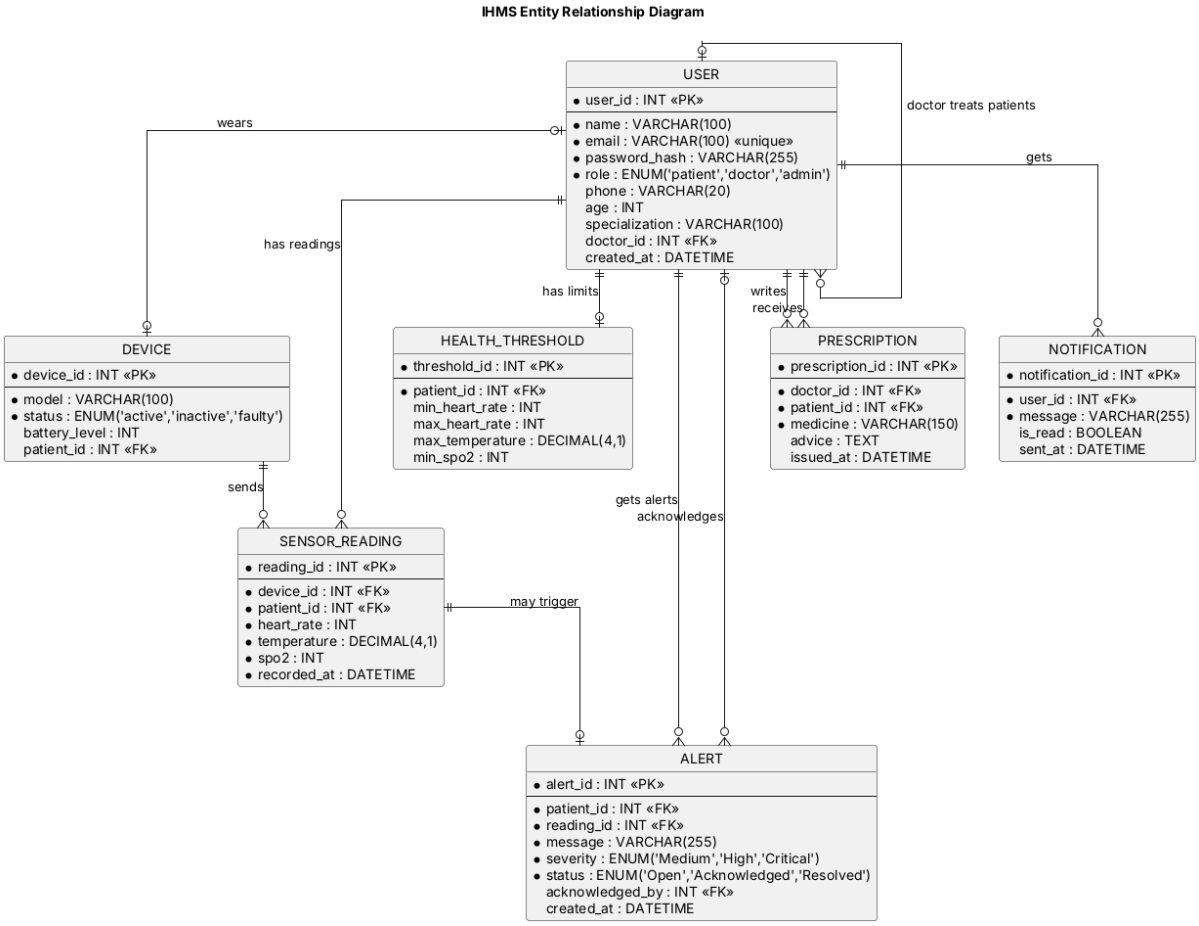


Figure 6: Entity relationship diagram

13.6 Class Diagram

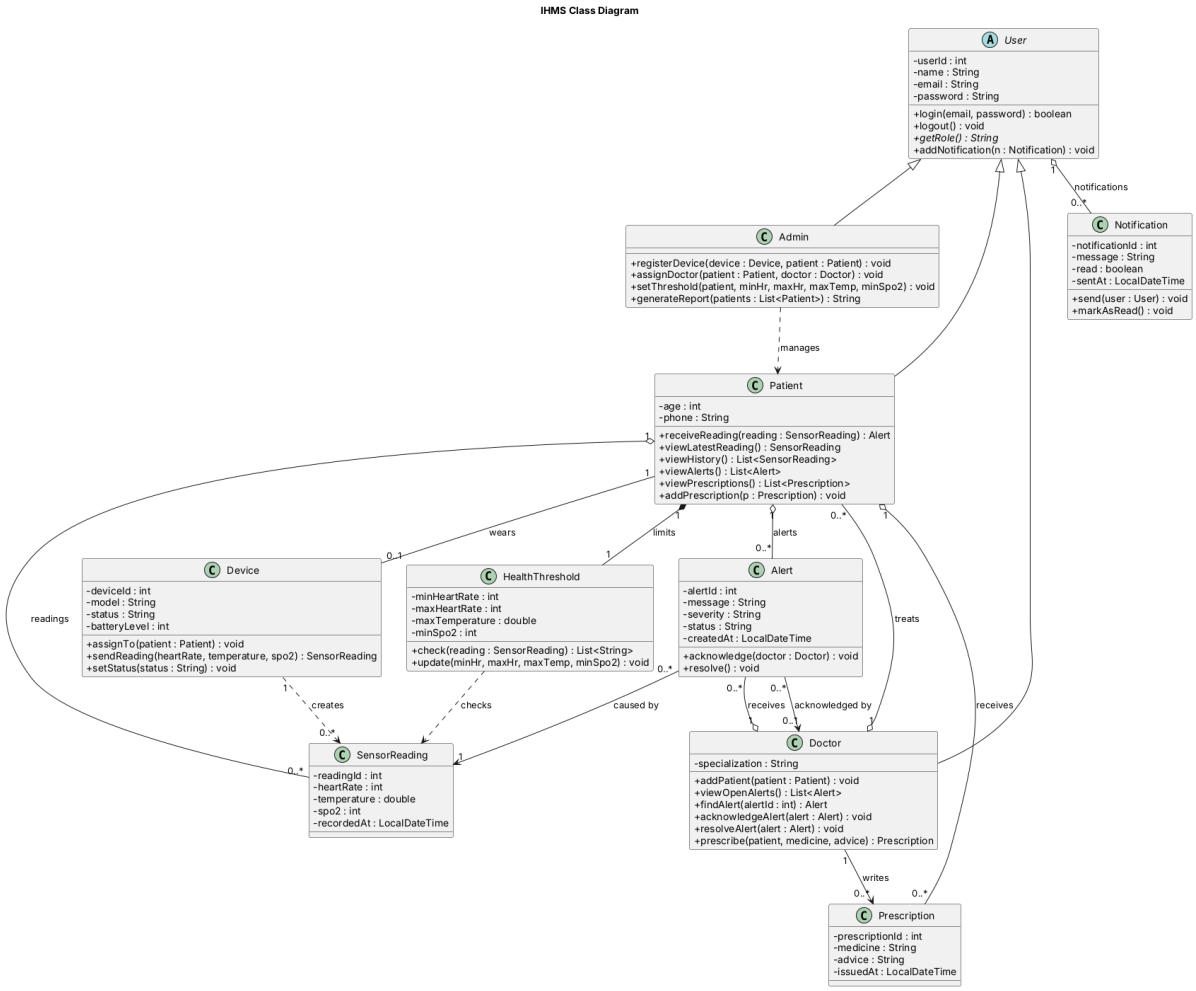


Figure 7: Class diagram

13.7 Object Diagram

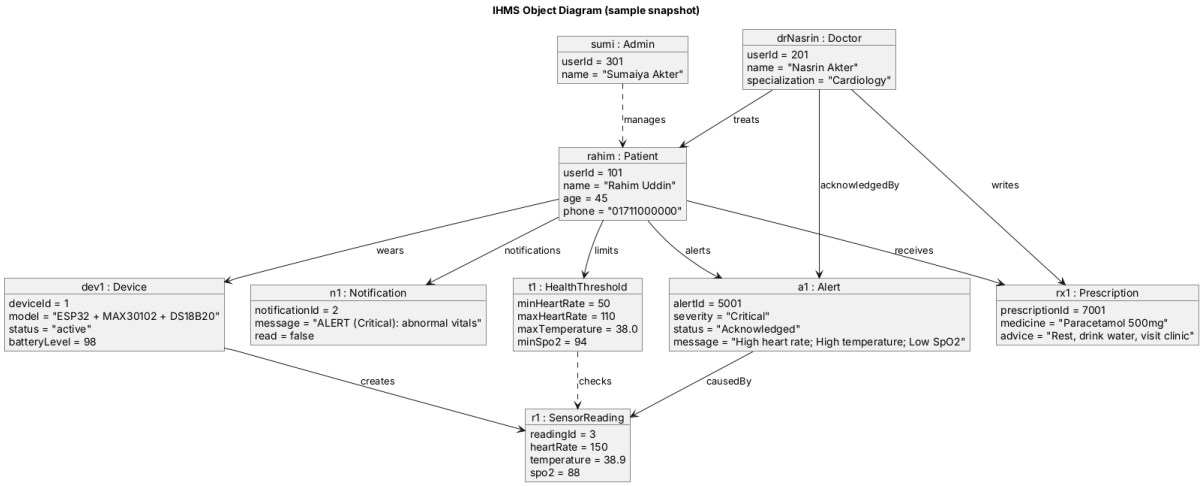


Figure 8: Object diagram (sample snapshot)

14 Prototype

A console prototype written in Java is provided in the `prototype/` folder. It implements the following ten classes from the class diagram, each in its own file: `User`, `Patient`, `Doctor`, `Admin`, `Device`, `SensorReading`, `HealthThreshold`, `Alert`, `Prescription` and `Notification`. The interface itself is in `Main.java`. The wearable device is simulated: the user types the heart rate, temperature and SpO_2 that the sensor node would send.

```
cd prototype
javac *.java
java Main           # interactive menu
java Main --demo    # automatic demo
```

The interface has four menus (Patient, IoT Device, Doctor, Admin) that cover sending readings, automatic alerts, doctor review, prescriptions, health limits and reports.

15 Conclusion

IHMS will allow continuous and remote monitoring of vital signs and give doctors and families a quick warning when something is wrong. This proposal defines the problem, requirements, schedule, design and a working prototype that will be extended with the real ESP32 sensor node and web dashboard in the next phase.